CPE 314 — Software Design & Software Design Laboratory

Week 8 — Project Proposal

Due end of Week 8 session | 10% of the Finals Term Grade | Scored out of 100

Note: this proposal is now graded — 10% of the Finals Term Grade, scored out of 100 — see Week08_Checklist.docx for the full Student Self-Check and Instructor Grading breakdown.

Group Name / No.:	***Group 3***
Members:	*Shann Michael Agang*
	Sofia Yvonne Enovero
	Kyle Cheznie Taghoy
	Aeisha Mae Elizar
	Cleint Lausa
Repository URL:	https://github.com/svgenov/cpe314--69323--capstone-team-03
Chosen Theme (circle one):	A B C D E F G H I J K L M N O (P) Q R S T ☐ Custom (attach approval)
API You Will Use:	— *https://dog.ceo/dog-api/documentation* —*https://thecatapi.com*

See Project_Specification_and_Rubric.docx, Section 4, for what each letter (A-T) means and Section 4.1 for the matching verified public API and its auth requirement.

Reminder: no two groups in this class may build the same theme — confirm your choice isn't already taken before submitting.

1. One-Paragraph Pitch

The Pet Shelter & Adoption Tracker is an offline-first Java desktop application designed to help animal shelter staff and volunteers transition away from scattered spreadsheets and paper records into a centralized system. The app streamlines daily operations by modeling species with custom care schedules, managing adoption waitlists using queues, supporting status undo actions via a stack, and enabling historical log imports from CSV files. Integrated pet APIs (Dog CEO API and TheCatAPI) auto-fill sample images and breed details, while built-in analytics render adoption trends and stay durations as visual bar charts. Ultimately, the system empowers shelter teams to make data-driven decisions and efficiently manage animals from intake through final adoption.

2. Entity-Relationship Sketch

(List your entities, their key attributes, and how they relate — text notation is fine, e.g., Member(id, name) --1:M--> Loan(id, memberId FK, bookId FK) <--M:1-- Book(id, title))

Entity	Key Attributes
Classification	id (PK), name, parent_id (FK → Classification.id)
Animal	id (PK), name, species, breed, age_months, intake_date, status, image_url classification_id (FK → Classification.id)
Adopter	id (PK), name, email (unique), phone
AdoptionRecord	id (PK), animal_id (FK → Animal.id), adopter_id (FK → Adopter.id), adoption_date, fee_paid
WaitlistEntry	id (PK), animal_id (FK → Animal.id), adopter_id (FK → Adopter.id), requested_at, position

Parent Entity	Relationship	Child Entity	Foreign Key	Description
Classification	1:M	Classification	parent_id (nullable)	Self-referencing taxonomy tree of any depth, e.g. Dog > Sporting Group > Retriever > Golden Retriever
Classification	1:M	Animal	classification_id	Each animal belongs to one classification
Animal	1:M	AdoptionRecord	animal_id	An animal can have adoption records
Adopter	1:M	AdoptionRecord	adopter_id	An adopter can adopt multiple animals
Animal	1:M	WaitlistEntry	animal_id	An animal can have many adopters waiting
Adopter	1:M	WaitlistEntry	adopter_id	An adopter can wait on multiple animals

3. Feature Checklist

Check off which mandatory requirement each planned feature will satisfy:

Planned Feature	Requirement(s) It Satisfies
Animal types: Dog, Cat, Other (one abstract class + three concrete classes)	**Object-Oriented Design**: Creates a main Animal blueprint with specialized sub-types that calculate their own adoption fees and care schedules.
Queue waitlist per animal (first person in line gets served first)	**Data Structure / Recursion:** a Queue (first-come, first-served line) for people waiting to adopt
Stack to undo last status change	**Data Structure / Recursion:** a Stack, like a Ctrl+Z for status changes, powering the “Undo Last Action” button
Tree for breeds (Dog > Sporting Group > Retriever > Golden Retriever) searched with recursion	**Data Structure / Recursion:** a search that digs through multi-level breed categories no matter how many levels deep
JUnit 5 tests (at least 6): fee calculation per subtype, queue order, undo on an empty stack, recursive search at shallow and deep levels	**Automated Testing:** normal and edge cases for the core logic classes
SQLite database with PreparedStatement for animals, adopters, and adoption records	**Data Modeling + Database:** Saving our shelter’s animals, adopters, and adoption history locally in an SQLite database using Java's standard database tools (JDBC).
Call Dog CEO API and TheCatAPI, read JSON, use images/breeds in app	**REST API Consumption** — live public API, with the parsed data shown in the app
Swing GUI with buttons and combo boxes (list animal, import CSV, record interest, mark adopted, view analytics)	**Event-Driven GUI:** Building an easy-to-use visual window with clickable buttons, forms, and lists so shelter staff don't have to use commands.
Import historical intake/adoption CSV on a background thread (e.g. SwingWorker)	**Concurrency:** the long-running import runs off the UI thread so the window never freezes
Analyze most-adopted breed/species and average shelter stay length	**Data Mining:** aggregation and frequency analysis over the shelter’s data
Bar chart of the mining results, with labeled axes and title	**Data Visualization:** a correctly labeled chart of the data mining output

4. Task Split

Member	Primary Responsibility (this milestone)
Sofia Yvonne Enovero	OOD model + class hierarchy, Queue/Stack, recursive tree search
Shann Michael Agang	Swing GUI + event handlers
Cleint Lausa	SQLite / JDBC data layer & DAOs
Kyle Cheznie Taghoy	REST API client + JSON parsing + CSV parsing/validation
Aeisha Mae Elizar	Concurrency (import/API workers) + analytics & chart

Note: Each person writes tests for their own module.

5. Public API You Plan to Use

(Name + one sentence on what data it provides)

- *https://dog.ceo/dog-api/documentation - Provides free dog images and breed lists without needing an API key.*
- *https://thecatapi.com - Provides cat images and breed details, but most features require a free API key.*

6. Risks / Open Questions

(Anything you're unsure how to implement yet — flag it now so the instructor can help during Week 8-9 consultation time)

- If Ever the API is down, How do we Control The flow?
- Should the SQLite db file be saved in the project folder, or somewhere else?
- Do the JUnit 5 tests need to run without a database connection or network access?